Final Project

Caiden Campeau

MSP-EXP430FR2355

07/22/2026

1. Problem Description

In this project, the goal is to design a system that allows the MSP430 to control the speed of a 12V DC motor using Bluetooth. Instead of using the matrix keypad like the last project, this new system uses the DSD Tech app on an iPhone to send commands to the HM-10 Bluetooth module. The module then communicates with the MSP430 using UART communication.

The MSP430 receives the command from the module and converts it to PWM duty cycle which is sent from pin P6.1 to the ENA pin on the motor driver which controls the motor speed. The driver is powered by a 9V battery which also powers the motor.

Also used in this system is the 2x16 LCD display connected with I2C communication. The LCD displays the motor speed percentage from 0% to 100%. The MSP430 also sends messages back to the app using UART so the phone displays the current motor speed after each input.

2. Pseudocode

Start program

Stop watchdog timer

Set up motor pwm

Set P6.1 as Timer_B PWM out

Set PWM period

Start PWM at 0

Set up I2C LCD

Set P4.6 as SDA

Set P4.7 as SCL

Initialize LCD display

Start motor speed on display at 0

Set up Bluetooth

Set P1.6 as UART RX

Set P1.7 as UART TX

Set baud to 9600

Enable UART receive

Send ready message to phone

Loop forever

Wait for Bluetooth command

When received

If command is 0 to 9

Convert to 0% to 90%

If command is A

Set speed to 100%

Update PWM duty cycle

Updated LCD display

Send message to phone

Repeat

3. C Code

```
#include <msp430.h>                    // include msp430 file
#define LCD_ADDR 0x27                  // lcd address
```

```
#define IN1 BIT0                            // motor input one
#define IN2 BIT1                            // motor input two

#define LCD_BACKLIGHT 0x08                  // lcd backlight bit
#define LCD_ENABLE    0x04                  // lcd enable bit
#define LCD_RS        0x01                  // lcd data select

volatile unsigned int speed = 0;            // motor speed percent
volatile unsigned char update = 0;          // update flag

void setup_motor(void);                     // setup motor
void setup_lcd_i2c(void);                   // setup lcd i2c
void setup_bluetooth(void);                 // setup bluetooth

void set_speed(unsigned int percent);       // set pwm speed
void send_char(char c);                     // send uart char
void send_text(char *text);                 // send uart text
void send_speed(unsigned int percent);      // send speed app

void i2c_send(unsigned char data);          // send i2c byte
void lcd_pulse(unsigned char data);         // pulse lcd
void lcd_write4(unsigned char data, unsigned char mode); // write nibble
void lcd_send(unsigned char data, unsigned char mode);   // send lcd byte
void lcd_cmd(unsigned char cmd);            // lcd command
void lcd_data(unsigned char data);          // lcd character
void lcd_init(void);                        // start lcd
void lcd_print(char *text);                 // print text
void lcd_speed(unsigned int percent);       // print speed

int main(void)
{
    WDTCTL = WDTPW | WDTHOLD;               // stop watchdog timer

    setup_motor();                          // setup pwm motor
    setup_lcd_i2c();                        // setup i2c lcd
    setup_bluetooth();                      // setup bluetooth uart

    PM5CTL0 &= ~LOCKLPM5;                   // unlock gpio pins

    UCB1CTLW0 &= ~UCSWRST;                  // start i2c
    UCA0CTLW0 &= ~UCSWRST;                  // start uart

    UCA0IE |= UCRXIE;                       // enable uart receive
    __enable_interrupt();                   // enable interrupts

    __delay_cycles(50000);                  // startup delay

    lcd_init();                             // initialize lcd
    set_speed(0);                           // motor starts off
    lcd_speed(0);                           // display 0 percent

    send_text("bluetooth ready\r\n");       // app start message
    send_speed(0);                          // send first speed
```

```
    while(1)                                // main loop
    {
        if(update == 1)                     // new command received
        {
            update = 0;                     // clear flag

            set_speed(speed);               // update pwm
            lcd_speed(speed);               // update lcd
            send_speed(speed);              // update app
        }
    }
}

void setup_motor(void)
{
    P6DIR |= BIT1;                          // p6.1 output
    P6SEL0 |= BIT1;                         // p6.1 timer pin

    P3DIR |= IN1 | IN2;                     // direction outputs
    P3OUT |= IN1;                           // motor forward
    P3OUT &= ~IN2;                          // motor forward

    TB3CCR0 = 655;                          // pwm period
    TB3CCTL2 = OUTMOD_7;                    // reset set mode
    TB3CCR2 = 0;                            // start at 0%
    TB3CTL = TBSSEL_1 | MC_1 | TBCLR;       // aclk up mode
}

void setup_lcd_i2c(void)
{
    UCB1CTLW0 = UCSWRST;                    // hold i2c reset
    UCB1CTLW0 |= UCSSEL_3;                  // use smclk
    UCB1BRW = 10;                           // i2c speed

    UCB1CTLW0 |= UCMODE_3;                  // i2c mode
    UCB1CTLW0 |= UCSYNC;                    // sync mode
    UCB1CTLW0 |= UCMST;                     // master mode
    UCB1CTLW0 |= UCTR;                      // transmit mode

    UCB1I2CSA = LCD_ADDR;                   // lcd slave address
    UCB1CTLW1 |= UCASTP_2;                  // auto stop
    UCB1TBCNT = 1;                          // one byte

    P4SEL1 &= ~(BIT6 | BIT7);               // select i2c pins
    P4SEL0 |=  (BIT6 | BIT7);               // p4.6 sda p4.7 scl
}

void setup_bluetooth(void)
{
    UCA0CTLW0 = UCSWRST;                    // hold uart reset
    UCA0CTLW0 |= UCSSEL__SMCLK;             // use smclk

    UCA0BRW = 104;                          // 9600 baud
    UCA0MCTLW = 0;                          // simple baud setup
```

```
    P1SEL1 &= ~(BIT6 | BIT7);               // select uart pins
    P1SEL0 |=  (BIT6 | BIT7);               // p1.6 rx p1.7 tx
}

void set_speed(unsigned int percent)
{
    TB3CCR2 = (percent * TB3CCR0) / 100;    // set duty cycle
}

void send_char(char c)
{
    while((UCA0IFG & UCTXIFG) == 0);        // wait for tx
    UCA0TXBUF = c;                          // send character
}

void send_text(char *text)
{
    while(*text)                            // while text remains
    {
        send_char(*text);                   // send character
        text++;                             // next character
    }
}

void send_speed(unsigned int percent)
{
    send_text("speed = ");                  // send label

    if(percent == 100)                      // full speed
    {
        send_text("100");                   // send 100
    }
    else if(percent == 0)                   // zero speed
    {
        send_text("0");                     // send 0
    }
    else                                    // 10 to 90
    {
        send_char((percent / 10) + '0');    // send tens digit
        send_char('0');                     // send zero
    }

    send_text("%\r\n");                     // send percent
}

void i2c_send(unsigned char data)
{
    while(UCB1CTLW0 & UCTXSTP);             // wait for stop

    UCB1TBCNT = 1;                          // send one byte
    UCB1CTLW0 |= UCTR;                      // transmit mode
    UCB1CTLW0 |= UCTXSTT;                   // start condition

    while((UCB1IFG & UCTXIFG0) == 0);       // wait tx buffer
    UCB1TXBUF = data;                       // send byte
```

```
    while(UCB1CTLW0 & UCTXSTP);             // wait for stop
}

void lcd_pulse(unsigned char data)
{
    i2c_send(data | LCD_ENABLE);            // enable high
    __delay_cycles(1000);                   // short delay
    i2c_send(data & ~LCD_ENABLE);           // enable low
    __delay_cycles(1000);                   // short delay
}

void lcd_write4(unsigned char data, unsigned char mode)
{
    unsigned char value;                    // lcd byte

    value = (data & 0xF0) | LCD_BACKLIGHT | mode; // build byte
    lcd_pulse(value);                       // send nib
}

void lcd_send(unsigned char data, unsigned char mode)
{
    lcd_write4(data, mode);                 // high nibble
    lcd_write4(data << 4, mode);            // low nibble


void lcd_cmd(unsigned char cmd)
{
    lcd_send(cmd, 0);                       // send command
    __delay_cycles(3000);                   // command delay
}

void lcd_data(unsigned char data)
{
    lcd_send(data, LCD_RS);                 // send data
}

void lcd_init(void)
{
    __delay_cycles(50000);                  // lcd delay

    lcd_write4(0x30, 0);                    // wake lcd
    __delay_cycles(50000);                  //
    lcd_write4(0x30, 0);                    // wake lcd
    __delay_cycles(50000);                  //
    lcd_write4(0x30, 0);                    // wake
    __delay_cycles(50000);                  // w

    lcd_write4(0x20, 0);                    // four bit mode
    __delay_cycles(50000);                  // wait

    lcd_cmd(0x28);                          // two lines
    lcd_cmd(0x0C);                          // display on
    lcd_cmd(0x06);                          // cursor moves right
    lcd_cmd(0x01);                          // clear display
```

```
    __delay_cycles(50000);                  // clear delay
}

void lcd_print(char *text)
{
    while(*text)                            // while text remains
    {
        lcd_data(*text);                    // print character
        text++;                             // next character
    }
}

void lcd_speed(unsigned int percent)
{
    lcd_cmd(0x01);                          // clear lcd
    __delay_cycles(50000);                  // clear delay

    lcd_cmd(0x80);                          // first line
    lcd_print("Motor Speed");               // print title

    lcd_cmd(0xC0);                          // second line

    if(percent == 100)                      // full speed
    {
        lcd_print("100%");                  // print 100
    }
    else if(percent == 0)                   // zero speed
    {
        lcd_print("0%");                    // print 0
    }
    else                                    // 10 to 90
    {
        lcd_data((percent / 10) + '0');     // print tens
        lcd_data('0');                      // print zero
        lcd_data('%');                      // print percent
    }
}

#pragma vector=EUSCI_A0_VECTOR
__interrupt void EUSCI_A0_ISR(void)
{
    char command;                           // bluetooth command

    command = UCA0RXBUF;                    // read command

    if(command >= '0' && command <= '9')    // number command
    {
        speed = (command - '0') * 10;       // make percent
        update = 1;                         // update system
    }
    else if(command == 'A' || command == 'a') // full speed
    {
        speed = 100;                        // set 100%
        update = 1;                         // update system
    }
```

```
else if(command == 'B' || command == 'b') // stop command
{
    speed = 0;                          // set 0%
    update = 1;                         // update system
}
}
```

4. Wiring Diagram

HM-10 Bluetooth Pin	MSP430 Pin / Connection	Purpose
VCC	HM-10 power pin used in build	power
GND	common ground rail	ground
TXD	P1.6	transmit to MSP430 RX
RXD	P1.7	receive from MSP430 TX

LCD Setup

LCD I2C Pin	Connect To	Purpose
GND	common ground rail	lcd ground
VCC	MSP430 3.3V	lcd power
SDA	P4.6	i2c data
SCL	P4.7	i2c clock

Motor Driver

L298N Pin	Connect To	Purpose
ENA	P6.1	pwm speed control
IN1	P3.0	motor direction
IN2	P3.1	motor direction
OUT1	motor wire 1	motor output
OUT2	motor wire 2	motor output
GND	common ground rail	shared ground
12V	9V battery positive	motor power

9V Battery and Power

Battery / Power	Connect To	Notes
9V battery positive	L298N +12V	motor power only
9V battery negative	common ground rail	shared ground
MSP430 USB	computer / USB power	powers MSP430
MSP430 GND	common ground rail	connects all grounds
MSP430 3.3V	LCD VCC	lcd power

5. YouTube Video

https://youtube.com/shorts/Rp66Ol3cZnw?feature=share

6. Conclusion

My system was successful in this project. I used the MSP430 to control a DC Motor wirelessly with the DSD Tech app and an HM-10 Bluetooth module. The module sent UART commands to MSP430 and was converted into PWM duty cycles from 0% to 100%. The L298N motor driver allowed the MSP430 to control the DC motor using the separate 9V power supply. The I2C LCD displayed the selected speed and sent the message back to the app. This project displayed how Bluetooth input, UART communication, PWM motor control and I2C LCD can all work together to create an embedded system. Thanks for the great semester.